

SecFlow

Repository Analysis Service

Phase 1 - Documentation & Reference

وثيقة مرجعية للـ approved architecture والتنفيذ الأولي للـ service

PHASE 1

GitHub Repository -> Repository Analysis Service -> repository_snapshot.yaml
-> AI Analyzer -> JSON Manifest

Status: Approved for implementation

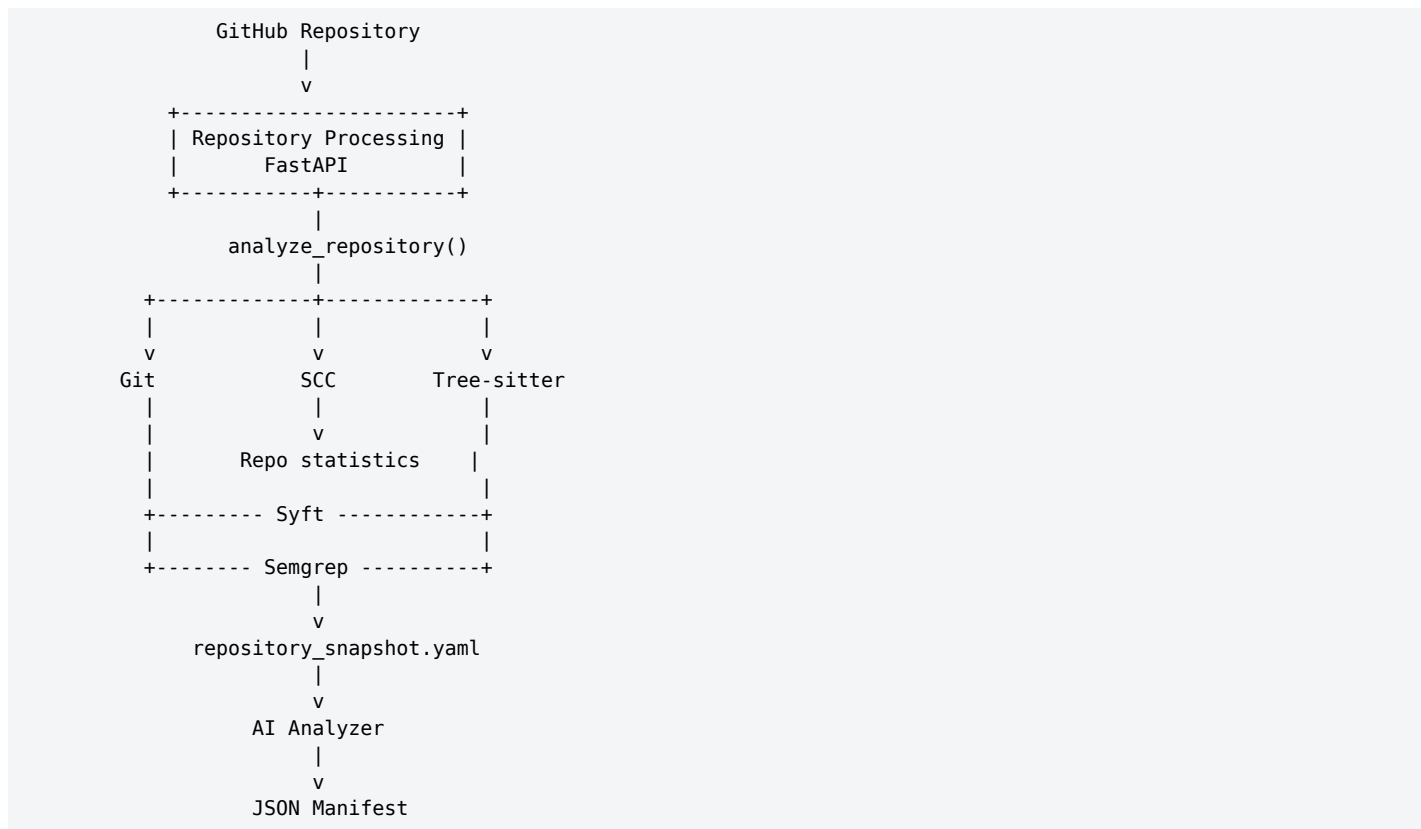
Scope: Repository intake, deterministic analysis, structured snapshot, AI analysis, and team ownership for this phase
.only

Future work such as RAG is intentionally خارج نطاق التنفيذ الحالي، ويُترك لمرحلة لاحقة بعد نجاح الـ MVP.

1. Phase 1 Overview

الفكرة الأساسية: نستقبل GitHub repository من الـ user، نعمل clone و deterministic processing و نبني structured repository snapshot بصيغة YAML، ثم نمرره إلى AI Analyzer لإنتاج JSON Manifest.

Primary Flow

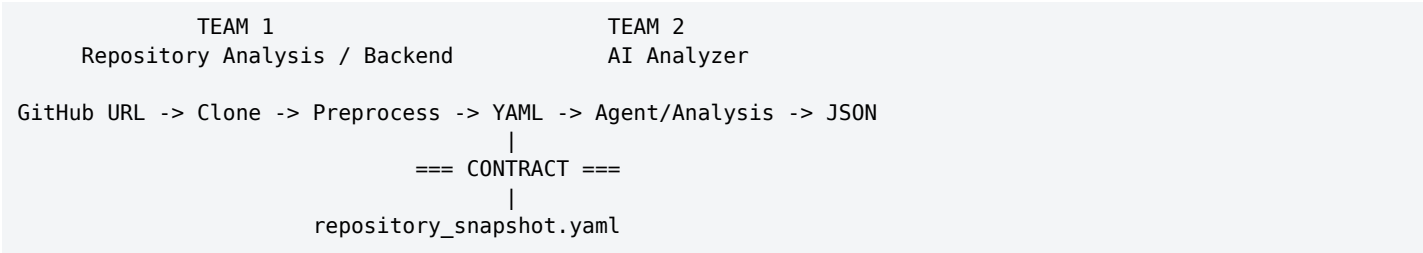


Approved Design Rules

- الـ Repository handling والـ preprocessing deterministic؛ لا يوجد AI داخل هذه الخطوة.
- الـ AI Analyzer يبدأ بعد وصول repository_snapshot.yaml.
- الـ YAML هو internal artifact بين الـ services، بينما الـ HTTP API يستخدم JSON.
- الـ AI لا يستخدم generic command execution؛ الـ orchestration function هي التي تستدعي أدوات Open Source controlled.
- RAG ليس جزءًا من الـ MVP الحالي، ويُعامل كـ future step.

2. Service Boundary & Data Contracts

Service Boundary



Team 1 owns repository lifecycle and deterministic inspection. Team 2 owns reasoning, AI orchestration, and **القاعدة المعتمدة**: .manifest generation

Repository Snapshot - YAML

```
repository:
  name: sample-app
  url: https://github.com/example/sample-app
files:
  - path: app/main.py
    type: source
    language: python
    content: "..."/>
  - path: Dockerfile
    type: infrastructure
    content: "..."/>
structure:
  directories:
    - app
    - tests
analysis:
  languages: {}
  dependencies: []
  patterns: []
```

الـ snapshot مش raw dump فقط؛ هو structured representation للـ repository، مع filtering و metadata و tool results حسب الحاجة.

AI Output - JSON Manifest

```
{
  "project": {"name": "sample-app", "type": "web_application"},
  "technologies": [
    {"name": "Python", "category": "language"},
    {"name": "FastAPI", "category": "framework"}
  ],
  "architecture": {"type": "monolith"},
  "dependencies": [],
  "deployment": {"containerized": true},
  "evidence": ["requirements.txt", "Dockerfile"]
}
```

3. Repository Analysis Service - FastAPI Microservice

الـ service دي تعتبر Microservice مستقلة داخل SecFlow، ومطلوب منها أن تكون deployable و testable وليها clear API contract.

API Contract

```
POST /api/v1/repositories/analyze
Content-Type: application/json

Request
{
  "repo_url": "https://github.com/example/project",
  "analysis_depth": "standard"
}

Response
{
  "status": "success",
  "repository": {
    "name": "project",
    "url": "https://github.com/example/project"
  },
  "snapshot": {
    "format": "yaml",
    "location": "/workspace/..."
  }
}
```

Recommended Project Structure

```
repository-service/
|
+-- app/
|   +-- main.py
|   +-- api/routes.py
|   +-- schemas/
|       | +-- requests.py
|       | +-- responses.py
|   +-- services/
|       | +-- repository.py
|       | +-- analyzer.py
|       | +-- snapshot.py
|   +-- tools/
|       +-- git.py
|       +-- scc.py
|       +-- tree_sitter.py
|       +-- syft.py
|       +-- semgrep.py
|
+-- tests/
+-- Dockerfile
+-- requirements.txt
+-- README.md
```

FastAPI Rationale

FastAPI هو الاختيار المعتمد لهذه الـ Python microservice لأنه مناسب للـ JSON API، يدعم type hints و request/response validation، ويوفر OpenAPI/Swagger documentation ويسهل containerization والـ integration.

4. Open Source Analysis Tools & Orchestration

الـ function الأساسية ليست Tool؛ هي Orchestrator Function تستدعي أدوات موجودة أصلاً وتحول نتائجها إلى structured analysis.

Tool	الدور داخل الـ service	نوع الاستخدام
Git	Clone / repository metadata / revision information	Repository lifecycle
SCC	Language detection + repository statistics	Deterministic inspection
Tree-sitter	AST/syntax-level inspection واستنتاج للكود structure	Code structure
Syft	SBOM-style و Dependency / package inventory extraction	Dependencies
Semgrep	Pattern-based code/config inspection	Technology / pattern evidence

Orchestrator Function

```
def analyze_repository(repo_url: str, config: AnalysisConfig):
    repository = clone_repository(repo_url)

    languages = analyze_languages(repository)          # SCC
    syntax = analyze_ast(repository)                   # Tree-sitter
    dependencies = analyze_dependencies(repository)    # Syft
    patterns = analyze_repository_patterns(repository) # Semgrep

    snapshot = build_snapshot(
        repository=repository,
        languages=languages,
        syntax=syntax,
        dependencies=dependencies,
        patterns=patterns,
    )

    return snapshot
```

Tool Access & MVP Note

الـ tools هي controlled adapters؛ الـ AI لا يملك generic shell execution. الـ MVP يختبر pipeline على sample project صغير، مع validated structured output. RAG future step و filtering/normalization.

5. Team 1 - Repository Analysis / Backend

Team 1 مسؤول بالكامل عن الـ Microservice من لحظة استقبال GitHub URL حتى إنتاج repository_snapshot.yaml.

Member 1 - Repository & Git Engineer

- FastAPI project setup والـ API endpoint الأساسي.
- استقبال GitHub URL والـ analysis configuration.
- Git clone / repository validation / branch & error handling / timeouts
- Temporary workspace lifecycle والـ cleanup.
- Dockerization للـ service.

```
GitHub URL
-> Validate
-> Clone
-> Workspace
```

Member 2 - Repository Analyzer / Open Source Tools

- بناء wrappers/adapters للأدوات: Git, SCC, Tree-sitter, Syft, Semgrep.
- تشغيل الأدوات و parsing للـ outputs.
- توحيد النتائج في internal data structures.
- Error handling و controlled tool execution.

```
Repository
-> Open-source analysis tools
-> Structured analysis results
```

Member 3 - Snapshot / Security / Integration

- تجميع نتائج الـ repository والـ tools.
- Filtering rules و metadata normalization.
- بناء repository_snapshot.yaml والتحقق منه.
- Security boundaries عند التعامل مع untrusted repositories.
- Integration tests + API documentation

```
Raw Repository + Tool Results
->
  Repository Snapshot
->
  YAML
```

6. Team 2 - AI Analyzer & Final Integration

Team 2 يبدأ عند repository_snapshot.yaml وينتهي بـ validated JSON Manifest.

Member 4 - AI Agent / Orchestration Engineer

- AI Agent orchestration والـ workflow.
- Model integration والـ tool/function calling.
- Decision logic والـ state/retry handling.
- تحديد إمتى الـ Agent يحتاج معلومات إضافية من الـ available tools/results.

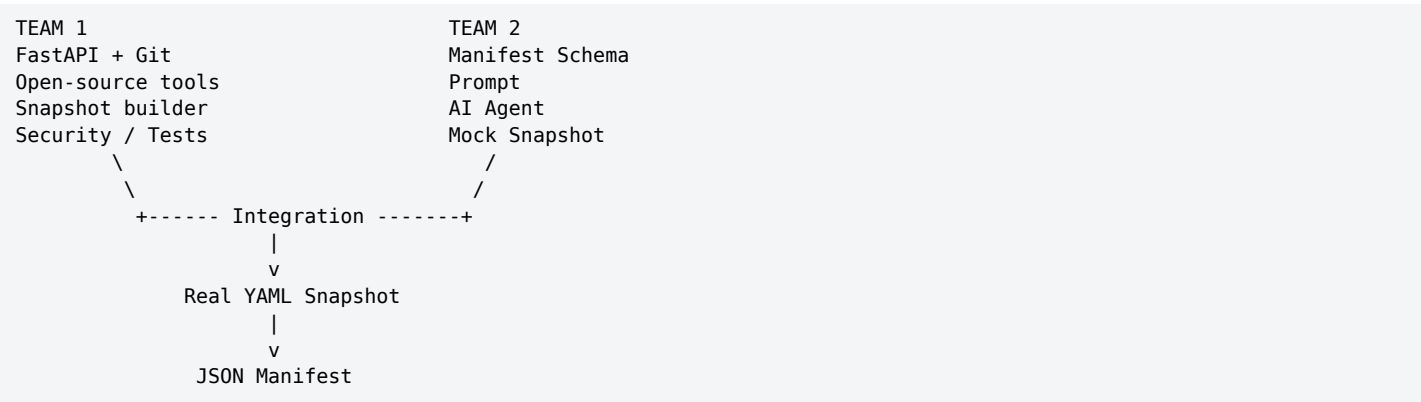
Member 5 - Prompt Engineering / Manifest Schema

- System prompt وتعليمات تحليل الـ repository.
- JSON Manifest schema والـ output constraints.
- Technology classification والـ architecture representation.
- Evidence / confidence / uncertainty handling.

Member 6 - AI Evaluation / Integration Engineer

- ربط Team 1 بـ Team 2 واستهلاك الـ snapshot.
- JSON validation والـ test cases.
- Sample projects والـ evaluation dataset.
- قياس accuracy/consistency of manifest generation وفشل الـ pipeline.
- Performance/cost testing للـ AI workflow.

Parallel Development Plan



Final Team Contract

Layer	Team 1	Team 2
Input	GitHub Repository URL	repository_snapshot.yaml
Main job	Fetch + inspect + normalize	Reason + infer + generate manifest
Primary artifact	repository_snapshot.yaml	JSON Manifest
AI responsibility	None in repository processing	Core analysis and orchestration
Boundary	YAML contract	Consumes YAML; produces validated JSON

ال approved boundary: Team 1 لا يدخل في AI reasoning، و Team 2 لا يدخل في Git clone أو repository preprocessing. كل فريق يقدر يطوّر ويختبر independently ثم يعمل integration عبر contract.

End of Phase 1 Reference